

Výstupy ŠVP

INF-05 Víme, co je algoritmus a znám typy programovacích jazyků. (Informatika 1, 1/1)

Učivo: algoritmizace – přirozené a formální jazyky, různé zápisy algoritmů

Základní pojmy

Přirozený jazyk – Čeština, angličtina...; bohatý a pružný, ale víceznačný („Viděl jsem muže s dalekohledem“).

Formální jazyk – Jazyk s přesně danými pravidly zápisu a významu – matematický zápis, programovací jazyky.

Syntaxe × sémantika – Syntaxe = pravidla, jak se smí psát; sémantika = co zápis znamená.

Vývojový diagram – Grafický zápis algoritmu ze značek spojených šipkami; dobře ukazuje větvení a cykly.

Pseudokód – Strukturovaný zápis blízký programu, ale bez přesné syntaxe konkrétního jazyka.

Proměnná a přiřazení – Pojmenované místo pro hodnotu; $x \leftarrow x + 1$ znamená „nová hodnota x je stará hodnota + 1“.

Podmínka – Výrok, který je pravdivý, nebo nepravdivý (H1); řídí větvení i cyklus.

Cyklus – Opakování: se známým počtem („opakuji 5×“, „pro každý prvek“) nebo s podmínkou („dokud ...“).

Překladač × interpret – Překladač (kompilátor) přeloží celý program předem do strojového kódu; interpret překládá a provádí program za běhu.

Nižší × vyšší jazyk – Nižší (strojový kód, assembler) je blízko procesoru; vyšší (Python, C, Java) je blízko lidskému myšlení.

Imperativní × deklarativní – Imperativní jazyk říká JAK (posloupnost příkazů – Python, C); deklarativní říká CO chci (SQL, Prolog).

Značky vývojového diagramu

Značka	Význam	Příklad
ovál	začátek / konec	Start, Konec
rovnoběžník	vstup / výstup	načti n ; vypiš součet
obdélník	příkaz, výpočet	$s \leftarrow s + n$
kosočtverec	rozhodnutí (podmínka) – dvě větve ano/ne	$n > 0?$
šipka	pořadí kroků; šipka zpět nahoru = cyklus	$\rightarrow$

Poučky a postupy

1. Tentýž algoritmus lze zapsat slovně (pro lidi), vývojovým diagramem (přehled větvení), pseudokódem (příprava na program) a programem (pro počítač).

2. Podmínka ve větvení i cyklu je výrok; spojky „a“, „nebo“, „ne“ fungují jako $\wedge$, $\vee$, $\neg$ z H1. „Pro každý prvek seznamu ...“ je cyklus – obdoba $\forall$.
3. Ve vývojovém diagramu je jeden začátek, aspoň jeden konec a z každého kosočtverce vedou právě dvě šipky (ano / ne).
4. V pseudokódu ukazuje odsazení, co patří do cyklu nebo větve.
5. Čtení cizího algoritmu: 1) urči vstup a výstup, 2) trasuj ho pro 2–3 vstupy (tabulka hodnot proměnných), 3) popiš jednou větou, co dělá, 4) zkontroluj konečnost a krajní případy.
6. Python a JavaScript se obvykle interpretují, C a C++ se překládají do strojového kódu; Java se překládá do mezikódu, který provádí virtuální stroj.

Pozor na

- Přiřazení $x \leftarrow x + 1$ není rovnice (ta by neměla řešení), ale příkaz.
- „Dokud platí P, opakuj“ × „opakuji, dokud neplatí P“: podmínky jsou opačné a druhý cyklus proběhne vždy aspoň jednou.
- HTML a CSS nejsou programovací jazyky (značkovací a stylový jazyk – nemají podmínky ani cykly).
- Kosočtverec s jedinou šipkou nebo větev, která nikam nevede, je chyba diagramu.

Vzorový příklad

Přečti cizí algoritmus (pseudokód): „vstup n (přirozené číslo); $c \leftarrow 0$; dokud $n > 0$: $n \leftarrow n // 10$ (celočíslné dělení), $c \leftarrow c + 1$; výstup c .“ Co dělá? Je to algoritmus?

1. Vstup: přirozené číslo n ; výstup: číslo c .
2. Trasování pro $n = 4071$: $n = 407$, 40, 4, 0 a $c = 1, 2, 3, 4 \rightarrow$ výstup 4.
3. Jednou větou: algoritmus spočítá, kolik číslic má číslo n .
4. Konečnost: n se v každém kroku zmenší (asi desetkrát) a je celé nezáporné $\rightarrow$ jednou bude 0.
5. Krajní případ $n = 0$: výstup 0, ale číslo 0 má jednu číslici $\rightarrow$ oprava: „je-li $n = 0$, vypiš 1“. Pak jde o jednoznačný, obecný a konečný algoritmus.